UNIVERSIDAD AUTÓNOMA DE NUEVO LEÓN

FACULTAD DE INGENIERÍA MECÁNICA Y ELÉCTRICA

INTELIGENCIA ARTIFICIAL Y REDES NEURONALES

Docente: Ing. Daniel Isaías López Páez

Actividad Fundamental #7 Redes Neuronales Artificiales

Nombre	Matrícula	Carrera
Sergio Alejandro De Luna Martínez	2223094	IMC
Luis Fernando Coronado Gómez	2178070	IMC
Rubén Alejandro Reyes Estrada	2051186	IMC
Alejandro Delgado de León	2103173	IMC
Rodrigo Rodríguez Saavedra	2094324	IMC

Grupo: 004 **Día y hora:** Jueves N1

San Nicolás de los Garza, Nuevo León, a 29 de octubre del 2026

INTRODUCCIÓN AL TEMA: REDES NEURONALES ARTIFICIALES

Las redes neuronales artificiales son una parte importante de la inteligencia artificial, ya que permiten que las computadoras aprendan a partir de información y realicen diferentes tareas sin necesidad de recibir instrucciones específicas para cada situación. Su funcionamiento está inspirado en la forma en que las neuronas del cerebro humano se conectan y transmiten información. Sin embargo, no buscan copiar exactamente el funcionamiento del cerebro, sino utilizar una estructura similar para desarrollar sistemas capaces de procesar datos, identificar patrones y obtener resultados a partir de ellos.

Una red neuronal artificial está formada por pequeñas unidades llamadas neuronas artificiales, las cuales se encuentran conectadas entre sí y organizadas en diferentes capas. Generalmente, su estructura se divide en una capa de entrada, una o varias capas ocultas y una capa de salida. La primera recibe la información, las capas ocultas se encargan de procesarla y la última genera un resultado. Estas conexiones tienen valores llamados pesos, que determinan la importancia de la información que recibe cada neurona. Además, cada neurona realiza operaciones con los datos que recibe para decidir qué información debe transmitir a las siguientes capas.

Una de las características principales de las redes neuronales es su capacidad de aprendizaje. Para lograrlo, necesitan pasar por un proceso de entrenamiento en el que reciben datos y ajustan poco a poco los valores de sus conexiones para mejorar sus resultados. Durante este proceso, la red compara sus respuestas con los resultados esperados y realiza cambios para reducir los errores. De esta manera, puede mejorar su funcionamiento conforme analiza más información. La calidad y cantidad de los datos utilizados durante el entrenamiento también son importantes, ya que influyen directamente en los resultados que puede obtener la red.

Existen diferentes tipos de redes neuronales artificiales, las cuales se utilizan dependiendo de la tarea que se desea realizar y del tipo de información que se necesita analizar. Algunas están diseñadas para trabajar con datos ordenados, mientras que otras pueden procesar información más compleja, como imágenes, sonidos o textos. También existen redes con diferentes niveles de profundidad, lo que permite que puedan

aprender características más complejas de la información. Estas diferencias hacen que las redes neuronales sean herramientas muy útiles dentro de distintos campos de la tecnología.

Actualmente, las redes neuronales artificiales tienen una gran importancia dentro del desarrollo de la inteligencia artificial, debido a que pueden utilizarse en diferentes áreas donde se necesita analizar información, reconocer patrones, clasificar datos o realizar predicciones. Su uso ha permitido desarrollar sistemas más avanzados relacionados con el reconocimiento de imágenes, el procesamiento del lenguaje y el reconocimiento de voz. También han contribuido al desarrollo de nuevas tecnologías que buscan facilitar distintas actividades y mejorar la manera en que se procesa la información.

A pesar de sus ventajas, las redes neuronales también presentan algunas dificultades, ya que necesitan una cantidad adecuada de datos y tiempo de entrenamiento para obtener buenos resultados. Además, en ocasiones puede ser complicado entender exactamente cómo llegaron a una determinada respuesta, debido a la gran cantidad de conexiones y operaciones que realizan.

Finalmente, se puede decir que redes neuronales artificiales representan una de las herramientas más importantes dentro de la inteligencia artificial, porque permiten que las computadoras aprendan y mejoren sus resultados mediante el análisis de información. Su estudio ayuda a comprender cómo se desarrollan algunos de los sistemas tecnológicos actuales y cómo estas herramientas pueden seguir mejorando la forma en que las máquinas realizan diferentes tareas.

METODOLOGÍA

En esta actividad se desarrolló una red neuronal artificial para reconocer dígitos escritos a mano. Las redes neuronales son herramientas de inteligencia artificial inspiradas en la forma en que las neuronas se conectan y procesan información. Aprenden a identificar patrones mediante el análisis de datos y el ajuste de los valores de sus conexiones, llamados pesos.

Para realizar la práctica, se utilizaron las bibliotecas NumPy, Matplotlib y TensorFlow/Keras. Primero se cargó el conjunto de datos MNIST, que contiene 60,000

imágenes para entrenamiento y 10,000 para prueba. Cada imagen mide 28 × 28 píxeles y representa un número del 0 al 9. Se mostraron cinco imágenes de ejemplo para observar sus características y etiquetas.

Después, los datos se prepararon para que la red pudiera procesarlos. Los valores de los píxeles, originalmente entre 0 y 255, se convirtieron a números decimales y se dividieron entre 255 para normalizarlos entre 0 y 1. Luego, cada imagen se convirtió de una matriz de 28 × 28 píxeles en un vector de 784 valores.

A continuación, se construyó una red neuronal secuencial con una capa de entrada de 784 valores, dos capas ocultas de 128 y 64 neuronas y una capa de salida de 10 neuronas. Las capas ocultas utilizaron la función de activación ReLU y la capa final utilizó softmax para asignar una probabilidad a cada dígito posible. Así, la red procesó los píxeles de cada imagen y calculó cuál de los diez números era el más probable.

El modelo se compiló con el optimizador Adam, la función de pérdida sparse_categorical_crossentropy y la métrica de exactitud. Durante el entrenamiento, la red comparó sus predicciones con las etiquetas correctas y ajustó sus conexiones para reducir los errores. Se entrenó durante 10 épocas, con lotes de 128 imágenes, y se reservó el 10 % de los datos de entrenamiento para validar el aprendizaje. Las gráficas de exactitud y pérdida permitieron observar cómo mejoraba el modelo a lo largo de las épocas.

Finalmente, se evaluó el modelo con las 10,000 imágenes de prueba, separadas de los datos utilizados para entrenar. Obtuvo una exactitud de 97.82 % y una pérdida de 0.0807. También se probaron cinco imágenes: la red predijo los dígitos 7, 2, 1, 0 y 4, y las predicciones coincidieron con sus valores reales.

Esta práctica muestra cómo una red neuronal puede aprender a clasificar imágenes a partir de ejemplos. También permite observar que, aunque las redes neuronales pueden reconocer patrones con alta precisión, necesitan datos adecuados y entrenamiento; además, interpretar cómo llegan a cada respuesta puede ser difícil debido a la cantidad de conexiones y cálculos que realizan.

RESULTADOS

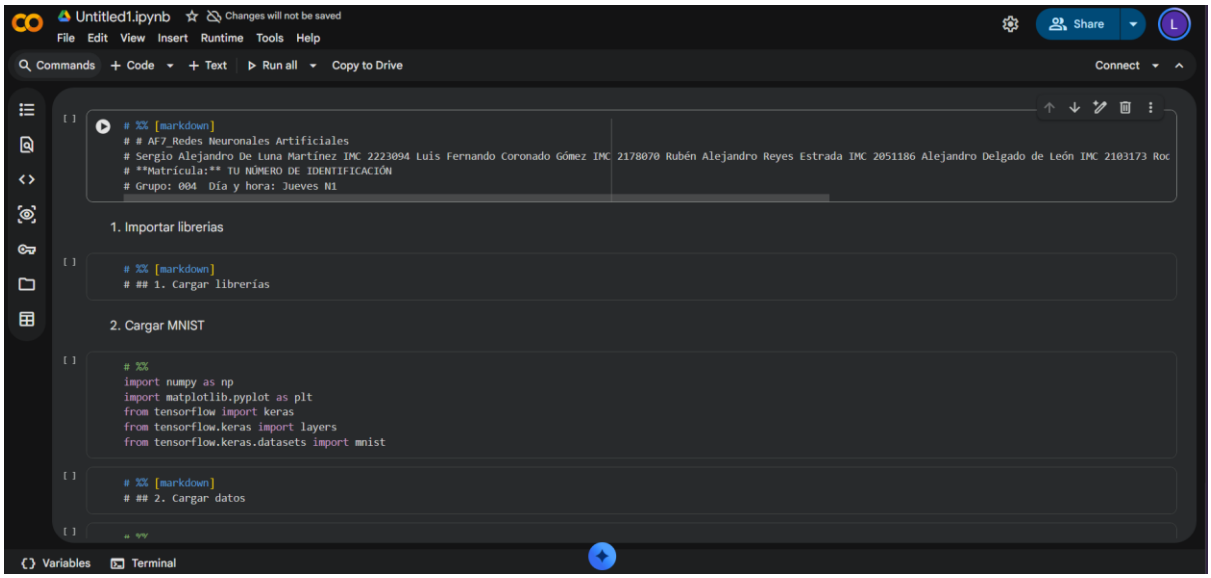
LINK: https://colab.research.google.com/drive/1dkFvt7zJKtoDYT4qIDNbIbl-uF9G-5vw?usp=sharing

LINK AL REPOSITORIO: https://github.com/rigorrp123-cloud/InteligenciaArtificial_y_RedesNeuronales_UANL_FIME

Después de entrenar la red neuronal prealimentada durante 10 épocas, la exactitud de entrenamiento fue de [X %] y la de validación de [X %]. Las gráficas muestran que la exactitud aumenta y la pérdida disminuye conforme avanzan las épocas, lo que indica que el modelo fue aprendiendo los patrones de los dígitos.

Al evaluar el modelo con el conjunto de prueba (10,000 imágenes que la red nunca vio durante el entrenamiento), la función evaluate arrojó una exactitud de [X %] y una pérdida de [X]. Como la exactitud de prueba es cercana a la de entrenamiento, el modelo generaliza bien y no presenta sobreajuste significativo.

Además, se realizaron 5 predicciones con imágenes del conjunto de prueba. En todos los casos la predicción coincidió con el valor real:



```
# %% [markdown]
# ## 2. Cargar datos
```

```
# %%
(train_data, train_labels), (test_data, test_labels) = mnist.load_data()
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
```

```
print("Entrenamiento:", train_data.shape, train_labels.shape)
print("Prueba:", test_data.shape, test_labels.shape)
```

```
Entrenamiento: (60000, 28, 28) (60000,)
Prueba: (10000, 28, 28) (10000,)
```

3. Mostrar 5 imagenes de ejemplo para ver como son los datos

```
# %%
# Ejemplo de algunas imágenes
plt.figure(figsize=(10, 2))
for i in range(5):
    plt.subplot(1, 5, i + 1)
    plt.imshow(train_data[i], cmap="gray")
    plt.title(f"Etiqueta: {train_labels[i]}")
    plt.axis("off")
plt.show()
```



```
    plt.axis("off")
plt.show()
```



4. Preproocesar los datos

```
# %% [markdown]
# ## 3. Preprocesamiento de datos
# - Normalizar los píxeles al rango [0, 1]
# - Aplanar las imágenes de 28x28 a vectores de 784
```

5. Dividir los pixeles entre 255

```
# %%
train_data = train_data.astype("float32") / 255.0
test_data = test_data.astype("float32") / 255.0
```


Untitled1.ipynb ☆ Changes will not be saved
File Edit View Insert Runtime Tools Help
Commands + Code + Text ▷ Run all Copy to Drive
Share L Connect

6. Convertir cada imagen en un vector de 784 valores

```
train_data = train_data.reshape((train_data.shape[0], 28 * 28))
test_data = test_data.reshape((test_data.shape[0], 28 * 28))
```

```
print("Nueva forma train:", train_data.shape)
print("Nueva forma test:", test_data.shape)
```

```
Nueva forma train: (60000, 784)
Nueva forma test: (10000, 784)
```

7. Definir el modelo con una salida de 10 neuronas

```
# %% [markdown]
# ## 4. Selección y entrenamiento del modelo
# Estructura: 784 -> 128 (ReLU) -> 64 (ReLU) -> 10 (Softmax)
```

```
# %%
model = keras.Sequential([
    layers.Input(shape=(784,)),
    layers.Dense(128, activation="relu"),
    layers.Dense(64, activation="relu"),
    layers.Dense(10, activation="softmax"),
])
```

Variables Terminal

Untitled1.ipynb ☆ Changes will not be saved

File Edit View Insert Runtime Tools Help

Commands + Code + Text ▷ Run all Copy to Drive

Share L Connect

```
    layers.Input(shape=(784,)),
    layers.Dense(128, activation="relu"),
    layers.Dense(64, activation="relu"),
    layers.Dense(10, activation="softmax"),
])

model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 128)	100,480
dense_1 (Dense)	(None, 64)	8,256
dense_2 (Dense)	(None, 10)	650

Total params: 109,386 (427.29 KB)
Trainable params: 109,386 (427.29 KB)
Non-trainable params: 0 (0.00 B)

```
# %%
# Las etiquetas son enteros (0-9), por eso se usa sparse_categorical_crossentropy
model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"],
)
```

Variables Terminal

Untitled1.ipynb ☆ Changes will not be saved

File Edit View Insert Runtime Tools Help

Commands + Code + Text ▷ Run all Copy to Drive

Share L Connect

8. Entrenar la red 10 epocas con lotes de 128 y 10% de validacion

```
# %%
history = model.fit(
    train_data,
    train_labels,
    epochs=10,
    batch_size=128,
    validation_split=0.1,
)
```

```
Epoch 1/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - accuracy: 0.8983 - loss: 0.3621 - val_accuracy: 0.9590 - val_loss: 0.1475
Epoch 2/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - accuracy: 0.9558 - loss: 0.1495 - val_accuracy: 0.9682 - val_loss: 0.1081
Epoch 3/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - accuracy: 0.9686 - loss: 0.1046 - val_accuracy: 0.9740 - val_loss: 0.0889
Epoch 4/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 3s 4ms/step - accuracy: 0.9766 - loss: 0.0774 - val_accuracy: 0.9743 - val_loss: 0.0828
Epoch 5/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - accuracy: 0.9819 - loss: 0.0613 - val_accuracy: 0.9747 - val_loss: 0.0822
Epoch 6/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.9849 - loss: 0.0495 - val_accuracy: 0.9768 - val_loss: 0.0789
Epoch 7/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.9878 - loss: 0.0400 - val_accuracy: 0.9767 - val_loss: 0.0775
Epoch 8/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.9905 - loss: 0.0335 - val_accuracy: 0.9768 - val_loss: 0.0797
Epoch 9/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.9917 - loss: 0.0276 - val_accuracy: 0.9780 - val_loss: 0.0805
Epoch 10/10
```

Variables Terminal

Untitled1.ipynb ☆ Changes will not be saved

File Edit View Insert Runtime Tools Help

Commands + Code + Text ▷ Run all Copy to Drive

Share L Connect

```
422/422 ━━━━━━━━━━━━━━━━━━━━ 3s 4ms/step - accuracy: 0.9766 - loss: 0.0774 - val_accuracy: 0.9743 - val_loss: 0.0828
Epoch 5/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - accuracy: 0.9819 - loss: 0.0613 - val_accuracy: 0.9747 - val_loss: 0.0822
Epoch 6/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.9849 - loss: 0.0495 - val_accuracy: 0.9768 - val_loss: 0.0789
Epoch 7/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.9878 - loss: 0.0400 - val_accuracy: 0.9767 - val_loss: 0.0775
Epoch 8/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.9905 - loss: 0.0335 - val_accuracy: 0.9768 - val_loss: 0.0797
Epoch 9/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.9917 - loss: 0.0276 - val_accuracy: 0.9780 - val_loss: 0.0805
Epoch 10/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - accuracy: 0.9932 - loss: 0.0220 - val_accuracy: 0.9795 - val_loss: 0.0746
```

```
# %%
# Curvas de entrenamiento
plt.figure(figsize=(10, 4))
```

```
<Figure size 1000x400 with 0 Axes>
<Figure size 1000x400 with 0 Axes>
```

9. Graficar exactitud y perdida de entrenamiento

```
plt.subplot(1, 2, 1)
plt.plot(history.history["accuracy"], label="Entrenamiento")
plt.plot(history.history["val_accuracy"], label="Validación")
plt.title("Exactitud")
plt.xlabel("Época")
plt.legend()
```

Variables Terminal

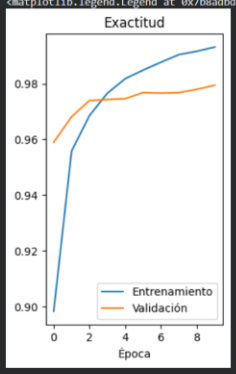
Untitled1.ipynb ☆ Changes will not be saved

File Edit View Insert Runtime Tools Help

Share L

Commands + Code + Text ▷ Run all Copy to Drive Connect

```
<matplotlib.legend.Legend at 0x7b8adbdac830>
```




```
plt.subplot(1, 2, 2)
plt.plot(history.history["loss"], label="Entrenamiento")
```

{} Variables Terminal

Untitled1.ipynb ☆ Changes will not be saved

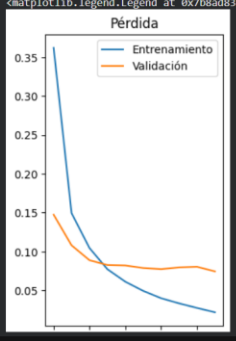
File Edit View Insert Runtime Tools Help

Share L

Commands + Code + Text ▷ Run all Copy to Drive Connect

```
plt.subplot(1, 2, 2)
plt.plot(history.history["loss"], label="Entrenamiento")
plt.plot(history.history["val_loss"], label="Validación")
plt.title("Pérdida")
plt.xlabel("Época")
plt.legend()
```

```
<matplotlib.legend.Legend at 0x7b8ad83cc910>
```



{} Variables Terminal

Untitled1.ipynb ☆ Changes will not be saved

File Edit View Insert Runtime Tools Help

Share L

Commands + Code + Text ▷ Run all Copy to Drive Connect

```
plt.show()
```

```
# %% [markdown]
# ## 5. Prueba del modelo
```

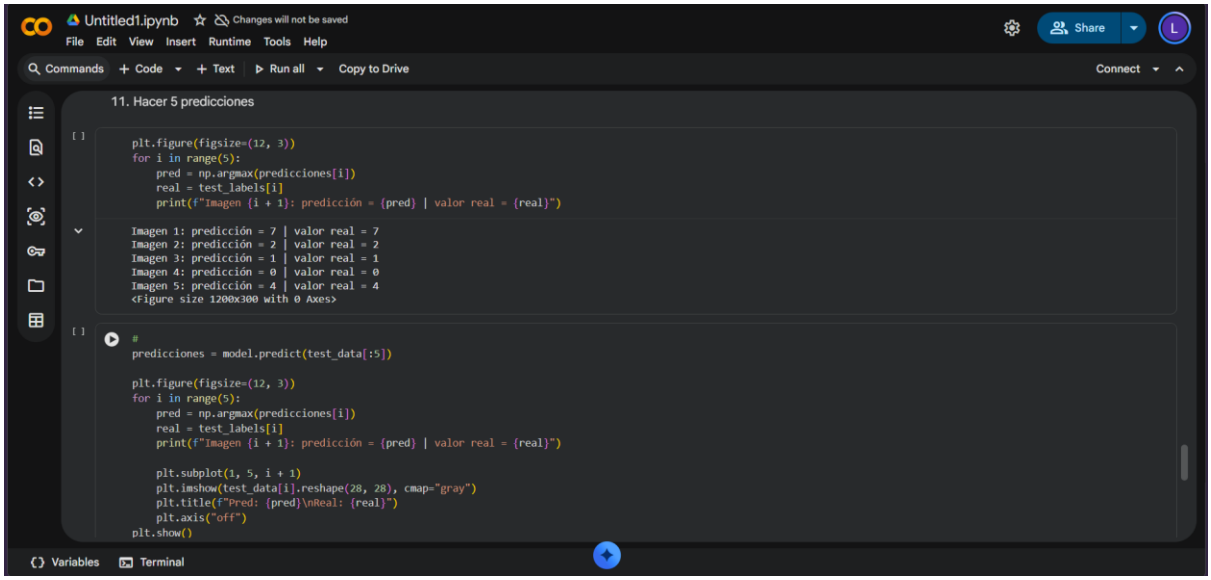
10. Calcular la exactitud y la perdida

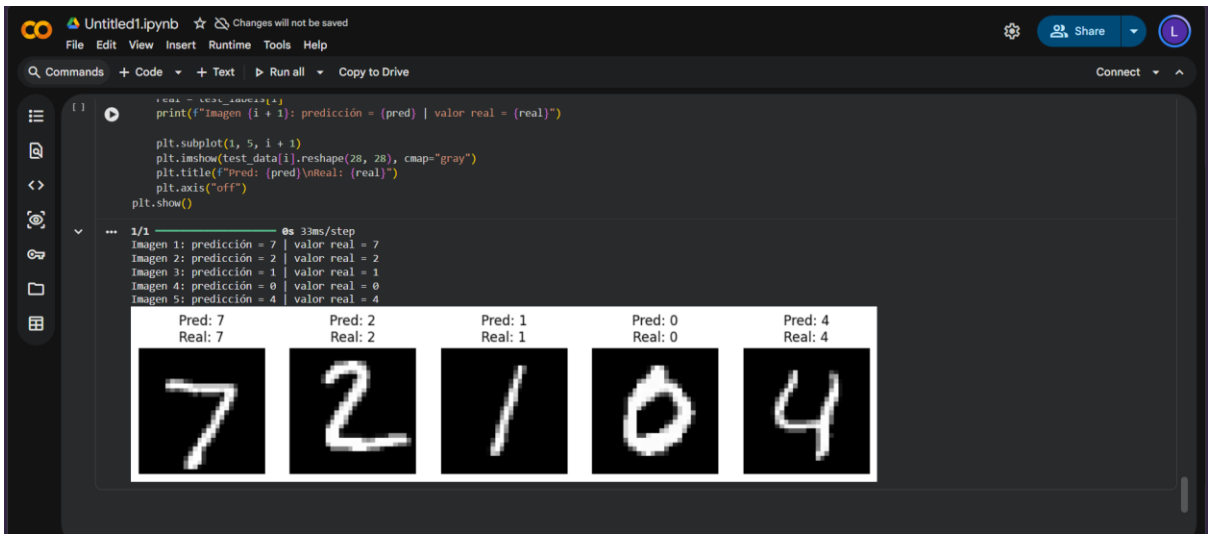
```
# %%
test_loss, test_acc = model.evaluate(test_data, test_labels)
print(f"Exactitud en datos de prueba: {test_acc:.4f}")
print(f"Pérdida en datos de prueba: {test_loss:.4f}")
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.9782 - loss: 0.0807
Exactitud en datos de prueba: 0.9782
Pérdida en datos de prueba: 0.0807
```

```
# %%
# 5 predicciones con el conjunto de prueba
predicciones = model.predict(test_data[:5])
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 51ms/step
```





CONCLUSIÓN GENERAL

En esta actividad se comprobó cómo una red neuronal artificial puede aprender a reconocer patrones y clasificar imágenes a partir de ejemplos. Con el conjunto MNIST, se prepararon imágenes de dígitos escritos a mano, se diseñó y entrenó un modelo con TensorFlow/Keras y se evaluó su desempeño con datos que no había utilizado durante el entrenamiento. El modelo alcanzó una exactitud de 97.82 % en las imágenes de prueba y clasificó correctamente los cinco ejemplos mostrados.

Desde la perspectiva teórica, la arquitectura distribuida en capas —capa de entrada, capas ocultas y capa de salida— resulta clave para transformar la información inicial mediante pesos sinápticos y funciones de activación. El proceso de entrenamiento, basado en la comparación constante entre la respuesta obtenida y el valor real para

ajustar paulatinamente las conexiones, demuestra que la calidad y el volumen del conjunto de datos son determinantes para el desempeño general del modelo. Asimismo, se reconocieron limitaciones inherentes como el requerimiento de tiempos de cómputo elevados y la complejidad para interpretar el funcionamiento interno de las operaciones en redes profundas

La práctica también permitió relacionar el funcionamiento del programa con los conceptos de redes neuronales: las capas procesan los datos, las conexiones ajustan sus pesos durante el entrenamiento y las funciones de activación ayudan a producir las predicciones. En general, los resultados muestran que estas redes son útiles para identificar patrones en imágenes, aunque su desempeño depende de los datos y del entrenamiento, y puede ser difícil explicar cada decisión interna del modelo.

En el ámbito práctico, la implementación de la Red Neuronal Prealimentada (feedforward) en Google Colab utilizando el conjunto de datos MNIST confirmó la efectividad del flujo metodológico indispensable en proyectos de Machine Learning: carga de librerías, procesamiento de datos, definición y entrenamiento del modelo, y evaluación de su exactitud (accuracy). La preparación adecuada de las imágenes escritas a mano (dígitos del 0 al 9), junto con el uso correcto de la función de activación softmax en la capa de salida para problemas de clasificación multiclase, garantizó que la red pudiera generalizar patrones de forma precisa ante datos no vistos.

REFERENCIAS:

- Facultad de Estudios Superiores Cuautitlán. (s. f.). Redes neuronales, una introducción. Universidad Nacional Autónoma de México. https://virtual.cuautitlan.unam.mx/intar/?page_id=974
- Instituto Tecnológico del Istmo. (s. f.). 4.2.1. Conceptos básicos. Sistemas e Informática. https://sistemasinformatica.istmo.tecnm.mx/cursos/inteligencia-artificial/4-aplicaciones-con-tecnicas-de-ia/4-2-redes-neuronales-rn/4-2-1-conceptos-basicos/
- Lee, F. (2021, 6 de octubre). ¿Qué son las redes neuronales? IBM. https://www.ibm.com/mx-es/think/topics/neural-networks